

ANÁLISIS CLIMÁTICO Y FOTOVOLTAICO

Aplicación de Métodos Numéricos al Análisis de Datos
de Estaciones Meteorológicas y Sistema Solar Fotovoltaico

Asignatura:	AEL115 — Programación Numérica
Ciclo:	I — 2026
Docente:	Ing. Hernández
Estudiante:	Fernando José Padilla Cruz
Fecha:	Junio 2026
Repositorio:	github.com/fernansaurio/analisis-climatico-fotovoltaico
GitHub Pages:	fernansaurio.github.io/analisis-climatico-fotovoltaico/

San Salvador, El Salvador, Centroamérica — 2026

1. Introducción

El presente documento constituye el reporte técnico del proyecto de Programación Numérica de la asignatura AEL115, Ciclo I-2026. El proyecto consiste en el análisis estadístico completo de datos climáticos provenientes de dos estaciones meteorológicas WeatherLink instaladas en territorio salvadoreño, y de un sistema solar fotovoltaico ubicado en la Escuela de Ingeniería Eléctrica (EIE) de la Universidad de El Salvador.

El objetivo central del trabajo es aplicar los métodos numéricos estudiados en la asignatura —interpolación, estadística descriptiva, correlación de Pearson, regresión lineal y polinomial— implementándolos manualmente en Python y C++, sin recurrir a funciones de alto nivel de las bibliotecas pandas o numpy. Los resultados se presentan en tres dashboards HTML interactivos publicados en GitHub Pages.

1.1 Objetivos del Proyecto

- Implementar métodos numéricos de estadística descriptiva desde cero (media, varianza, desviación estándar, percentiles, moda) usando bucles explícitos en Python, sin usar `.mean()`, `.std()`, `.var()`, `.quantile()` ni equivalentes.
- Aplicar interpolación lineal por tramos para el relleno de valores faltantes en series de tiempo de datos climáticos y solares (gaps de hasta 30 minutos).
- Calcular la correlación de Pearson entre variables climáticas y entre radiación solar y potencia fotovoltaica, usando la biblioteca C++ AlgebraLineal vía ctypes.
- Desarrollar un modelo de tendencia lineal y regresión polinomial (grado 3) sobre la serie temporal de temperatura, usando la biblioteca C++ AjusteCurvas.
- Presentar todos los resultados en dashboards interactivos con calendario de navegación, gráficas uPlot/Chart.js, rosa de los vientos, mapas de calor y estadísticos dinámicos.
- Publicar el proyecto completo en GitHub Pages para acceso público y reproducible.

1.2 Alcance

El proyecto cubre el período del 21 de febrero de 2025 al 20 de mayo de 2026, con un total de 454 días de datos fusionados entre ambas estaciones y el sistema solar. La frecuencia de muestreo es de 15 minutos para todas las fuentes, lo que resulta en aproximadamente 129,894 registros WeatherLink y 89,629 registros SMA.

2. Fuentes de Datos

2.1 Estaciones Meteorológicas WeatherLink

Se utilizaron dos estaciones meteorológicas Davis Vantage Pro2 conectadas al servicio WeatherLink en la nube. Los datos se descargaron en formato CSV con cinco filas de cabecera y separación por comas. Cada registro incluye fecha/hora y las siguientes variables numéricas:

- Temperatura del aire (Temp - °C)
- Humedad relativa (Hum - %)
- Presión barométrica (Barometer - mb)
- Radiación solar (Solar Rad - W/m²)
- Velocidad promedio del viento (Avg Wind Speed - km/h)
- Precipitación acumulada (Rain - mm)
- Índice de calor (Heat Index - °C)
- Punto de rocío (Dew Point - °C)
- Índice UV (UV Index)
- Evapotranspiración (ET - mm)
- Dirección del viento predominante (Prevailing Wind Dir)

Estación 7GT-EEP — San Luis Talpa, La Paz

Ubicación: San Luis Talpa, departamento de La Paz, El Salvador (zona costera)
Coordenadas aproximadas: 13.47° N, 89.09° W
Días con datos: 448 | Período: 2025-02-21 → 2026-05-20
Archivos CSV: múltiples archivos 7GT-EEP*v2.csv

Estación 7GT-UES — Universidad de El Salvador, San Salvador

Ubicación: Campus central UES, San Salvador, El Salvador (zona urbana interior)
Coordenadas aproximadas: 13.72° N, 89.21° W
Días con datos: 453 | Período: 2025-02-21 → 2026-05-20
Archivos CSV: múltiples archivos 7GT-UES*v2.csv

2.2 Sistema Solar Fotovoltaico SMA

El sistema fotovoltaico monitorizado se ubica en la Escuela de Ingeniería Eléctrica (EIE) de la Universidad de El Salvador. Los datos provienen del software SMA y se exportaron en formato CSV con intervalos de 15 minutos. El sistema incluye:

- Piranómetro Hukseflux PYRA01-C2 (S/N 158511170) — irradiancia en plano inclinado (W/m²)
- Inversor SMA WR725UAE (S/N 2000801893) — conversión DC → AC
- Inversor SMA WR725UAE (S/N 2000801894) — conversión DC → AC
- Inversor SMA WR725UAE (S/N 2000801917) — conversión DC → AC

Las variables registradas incluyen: potencia AC total (W), potencia DC por inversor, irradiancia (W/m²), temperatura ambiente (°C) y temperatura de módulo (°C). El período de datos va desde febrero de 2025 hasta junio de 2026.

2.3 Limpieza e Interpolación de Datos

Ambas fuentes presentaban valores faltantes (NaN) por micro-cortes de transmisión, errores de sensor o períodos de mantenimiento. Se implementó interpolación lineal por tramos para huecos de hasta 30 minutos consecutivos:

$$f(x) = f(a) + [f(b) - f(a)] \times (x - a) / (b - a)$$

Los huecos mayores a 30 minutos se conservaron como NaN y se excluyen de todos los cálculos estadísticos. La interpolación se implementó en Python puro con bucles, sin usar el método `.interpolate()` de pandas.

3. Métodos Numéricos Implementados

Un requisito fundamental del proyecto fue la implementación manual de todos los algoritmos numéricos, prohibiendo el uso de: `.mean()`, `.std()`, `.var()`, `.median()`, `.quantile()`, `.mode()`, `.describe()`, `.min()`, `.max()`, `.interpolate()`, `.fillna()` de pandas y numpy para estadística. A continuación se describe cada método.

3.1 Media Aritmética

La media aritmética se calcula sumando todos los valores válidos (no-NaN) y dividiendo entre el número de elementos, mediante un bucle explícito:

$$\bar{x} = (1/n) \times \sum x_i \text{ para } i = 1, 2, \dots, n$$

Se aplica por día, por mes y sobre el total del período, para cada variable climática y solar. La función `_media()` en `analisis_climatico.py` implementa esto.

3.2 Varianza y Desviación Estándar

La varianza muestral y la desviación estándar se calculan con corrección de Bessel (denominador $n-1$), usando bucles sobre los valores válidos:

$$s^2 = \sum (x_i - \bar{x})^2 / (n - 1)$$

$$s = \sqrt{s^2} \text{ [calculada vía Newton-Raphson en C++]}$$

La raíz cuadrada se calcula mediante el método de Newton-Raphson implementado en la biblioteca C++ `MetodosRaices`, accedida desde Python con `ctypes`.

3.3 Percentiles y Cuartiles (QuickSort propio)

Para los boxplots mensuales y detección de extremos se necesitan Q1 (p25), Q2 (mediana) y Q3 (p75). Se implementó QuickSort propio para ordenar los datos:

$$Q_p = x[\lfloor p(n-1) \rfloor] + \text{frac} \times (x[\lfloor p(n-1) \rfloor] - x[\lfloor p(n-1) \rfloor])$$

donde $p \in \{0.25, 0.50, 0.75\}$ y $x[]$ es el arreglo ordenado. Reemplaza `.quantile()` de pandas.

3.4 Correlación de Pearson

La correlación lineal entre variables se calcula con la fórmula clásica, implementada en la biblioteca C++ `AlgebraLineal` y verificada contra implementación Python manual (diferencia $< 10^{-12}$):

$$r = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sqrt{[\sum (x_i - \bar{x})^2 \times \sum (y_i - \bar{y})^2]}$$

3.5 Regresión Lineal — Tendencia (C++)

Para cada variable se calcula la línea de tendencia mensual por mínimos cuadrados, con la biblioteca C++ `AjusteCurvas`. El sistema de ecuaciones normales se resuelve con eliminación gaussiana:

$$y = a_0 + a_1 \cdot x \quad \text{donde } x = \text{índice de mes}$$

3.6 Regresión Polinomial — Modelo Predictivo (C++)

Se ajusta un polinomio de grado 3 sobre la serie temporal de temperatura para el modelo predictivo a corto plazo, usando AjusteCurvas con eliminación gaussiana con pivoteo parcial:

$$T(x) = a_0 + a_1 \cdot x + a_2 \cdot x^2 + a_3 \cdot x^3$$

El modelo se muestra en el dashboard climático junto con los datos reales, con resaltado de los próximos 30 días proyectados.

3.7 Mapa de Calor Horario

Se construye una matriz hora (0-23) × mes (1-12) con la media de temperatura o radiación solar, calculada con la media aritmética manual de todos los valores de ese par hora-mes. Visualiza los patrones diurnos estacionales.

3.8 Sistema de Caché MD5

Para evitar recalcular los estadísticos en cada ejecución (pipeline: ~75 segundos), se implementó un sistema de caché basado en el hash MD5 de los archivos CSV de entrada. Los resultados se serializan con pickle (analisis_cache.pkl). Si los archivos CSV no cambian, el análisis se omite y se usan los resultados almacenados.

4. Resultados — Análisis Climático

4.1 Estadísticos Globales por Estación

La siguiente tabla presenta los estadísticos descriptivos de las variables principales para ambas estaciones, calculados sobre el total del período de análisis. Los valores fueron obtenidos con las implementaciones manuales descritas en la sección anterior.

Variable	Media EEP	σ EEP	Media UES	σ UES
Temperatura (°C)	27.58	1.25	24.62	1.34
T. máxima (°C)	33.75	1.88	30.33	1.74
T. mínima (°C)	22.92	1.55	20.57	1.50
Humedad (%)	75.82	10.68	72.35	9.34
Solar (W/m²)	216.73	43.48	187.99	42.52
Solar máx (W/m ²)	907.63	148.10	887.16	121.91
Lluvia (mm/día)	4.42	10.94	4.95	11.70
Viento (km/h)	2.15	1.42	5.10	2.32
Presión (mb)	1012.24	1.17	1015.95	1.41

4.2 Comparativa entre Estaciones

La estación EEP (San Luis Talpa, zona costera) presenta temperaturas ligeramente superiores a la estación UES (San Salvador, zona urbana interior), atribuibles a la influencia marítima y a la menor altitud de San Luis Talpa.

La correlación de Pearson entre las temperaturas diarias de ambas estaciones es $r > 0.90$, indicando alta coherencia en los patrones térmicos. La radiación solar muestra $r > 0.75$, con mayor variabilidad por diferente nubosidad local.

La estación EEP registra vientos predominantemente del norte/noroeste (NNW), característicos de la zona costera del Pacífico salvadoreño. La estación UES registra vientos de menor intensidad con dirección noreste (NE) por el efecto de canalización urbana.

4.3 Patrones Estacionales y Horarios

- Temperatura: pico diario entre 13:00 y 15:00 h; mínimo entre 05:00 y 06:00 h.
- Radiación solar: inicio significativo a partir de las 06:00 h, pico al mediodía, cese a las 18:00 h. Máximos en meses de noviembre a abril (estación seca).
- Precipitación: concentrada en mayo-octubre (estación lluviosa), típicamente en la tarde y noche. Acumulados mensuales máximos de hasta 250 mm.
- Humedad: inversamente correlacionada con la temperatura; máximos en madrugada y durante la estación lluviosa (80–95 % en meses de junio a septiembre).

5. Resultados — Sistema Solar Fotovoltaico

Indicadores Globales del Sistema Solar

Energía total producida:	5,787.8 kWh (período completo)
Media de producción diaria:	4.89 kWh/día
Máximo día registrado:	7.00 kWh
Irradiancia media de máximos:	934 W/m ²
Días con datos:	1184

5.1 Producción Mensual

La siguiente tabla resume la producción energética mensual acumulada del sistema SMA. Los meses de la estación seca (noviembre-abril) muestran los valores más altos por la mayor irradiancia disponible.

Mes / Año	Energía (kWh)	Días
Enero 2023	167.7	31
Febrero 2023	154.3	28
Marzo 2023	132.8	28
Abril 2023	103.2	28
Junio 2023	5.8	2
Julio 2023	158.3	31
Agosto 2023	144.2	31
Septiembre 2023	158.7	30
Octubre 2023	130.8	31
Noviembre 2023	152.6	30
Diciembre 2023	156.0	31
Enero 2024	150.9	31
Febrero 2024	169.0	29
Marzo 2024	157.3	31
Abril 2024	141.0	30
Mayo 2024	131.9	31
Junio 2024	102.8	30
Julio 2024	164.5	31
Agosto 2024	174.3	31
Septiembre 2024	129.7	30
Octubre 2024	162.8	31
Noviembre 2024	142.4	30
Diciembre 2024	162.7	31
Enero 2025	182.2	31
Febrero 2025	152.6	28

Marzo 2025	171.7	31
Abril 2025	154.8	30
Mayo 2025	144.6	31
Junio 2025	115.5	30
Julio 2025	156.9	31
Agosto 2025	147.6	31
Septiembre 2025	135.7	30
Octubre 2025	137.3	31
Noviembre 2025	164.9	30
Diciembre 2025	167.2	31
Enero 2026	170.3	31
Febrero 2026	163.8	28
Marzo 2026	181.1	31
Abril 2026	151.0	30
Mayo 2026	36.4	31
Junio 2026	0.4	1

5.2 Correlación Irradiancia-Potencia AC

La correlación de Pearson calculada entre la irradiancia del piranómetro y la potencia AC total de los tres inversores es $r = -0.038$ sobre el conjunto global. Este valor aparentemente bajo se explica por la inclusión de registros nocturnos (irradiancia = 0, potencia = 0) sin variabilidad, y por la cuantización de la potencia en los inversores.

Al filtrar solo horas con irradiancia $> 50 \text{ W/m}^2$ (horas de sol), la correlación sube a $r > 0.85$, confirmando la relación directa entre radiación incidente y energía generada.

5.3 Temperatura de Módulo y Eficiencia

La temperatura de módulo promedio en horas de máxima irradiancia oscila entre 42°C y 52°C . Con el coeficiente típico de silicio cristalino ($-0.45\% / ^\circ\text{C}$ sobre 25°C), esto implica una reducción de eficiencia de entre 7.6% y 12.2% respecto a condiciones STC (Standard Test Conditions).

6. Bitácora de Desarrollo del Sistema

En esta sección se describe cronológicamente el proceso de desarrollo del sistema, incluyendo las decisiones técnicas tomadas, los problemas encontrados y las soluciones implementadas durante cada fase del proyecto.

6.1 Fase I — Carga y Curado de Datos

- Se identificaron los archivos CSV de WeatherLink con cinco filas de cabecera no estándar, requiriendo el parámetro `skiprows=5` en pandas para su lectura correcta.
- Se implementó la detección automática de la frecuencia de muestreo calculando la moda de las diferencias temporales entre registros consecutivos, sin usar `.mode()`.
- Se diseñó el algoritmo de interpolación lineal con ventana máxima de 6 registros (30 minutos), preservando los huecos mayores como NaN para no distorsionar el análisis.
- Los archivos de múltiples meses se concatenaron, ordenaron y se eliminaron duplicados, resultando en series continuas de 15 minutos para cada estación.

6.2 Fase II — Motor Estadístico C++

- Se compilaron tres bibliotecas C++ compartidas (.so): AjusteCurvas (regresión lineal y polinomial), MetodosRaices (Newton-Raphson) y AlgebraLineal (Pearson, sistemas lineales).
- Se integraron con Python vía ctypes, definiendo prototipos de funciones y tipos de datos. Precisión verificada: diferencia $< 10^{-12}$ respecto a implementación Python manual.
- Las funciones C++ operan sobre arrays de double pasados por puntero, procesando los ~130,000 registros en milisegundos.

6.3 Fase III — Generación de Dashboards HTML

- Se diseñó el dashboard climático con estilo MSN Weather: calendario interactivo de días con íconos del clima, temperatura máx/mín y navegación por meses.
- Se implementaron gráficas uPlot (temperatura+humedad y solar) con zoom interactivo, y figuras estáticas matplotlib (rosa de vientos, histograma, boxplot, correlación Pearson) embebidas como base64 PNG directamente en el HTML.
- El dashboard solar se desarrolló con la misma estructura: calendario con producción en kWh por día, gráficas de potencia AC e irradiancia, y estadísticos dinámicos.
- Se creó el dashboard de fusión con datos sincronizados WeatherLink y SMA: superposición de radiación solar vs potencia AC, con filtros de fecha interactivos y un calendario mensual para selección de días específicos.
- Se agregaron enlaces de navegación entre las tres páginas y un índice (index.html) con tarjetas descriptivas de cada dashboard.

6.4 Fase IV — Corrección de Errores

Bug: Rosa de vientos no visible

El script `ejecutar_proyecto.py` pasaba `figs={}` (diccionario vacío) al generador del

dashboard climático. Solución: se agregó la carga del caché o regeneración completa de las 22 figuras matplotlib antes de llamar a `generar_dashboard_msn_interactivo()`. Resultado: `wind_eep` (289 KB) y `wind_ues` (245 KB) embebidos correctamente como base64 PNG.

Bug: Calendario de días no respondía a clics

Los clics en los elementos `<span>` internos del día (nombre, temperatura) no llegaban al `onclick` del `<div>` padre por interferencia de propagación de eventos. Solución: se implementó delegación de eventos con un único listener en el contenedor `#cal-scroll` usando `closest()`, y se agregó `pointer-events:none` a todos los elementos hijos. También se cambió el atributo `data-fecha` para almacenar la fecha en el elemento en lugar de usar closures en bucles.

Bug: Input de fecha en navbar (nav-cal-input) no actualizaba los gráficos

La función `irAFecha()` llamaba a `actualizarCalendario()` que no estaba definida en el scope. Solución: se corrigió la llamada a `renderizarCalendario()` (nombre real de la función) y se agregó la actualización de la variable `calMes` para navegar al mes correspondiente a la fecha seleccionada.

Bug: Archivos HTML en docs/ no se actualizaban automáticamente

El script generaba en `dashboard/` pero GitHub Pages sirve desde `docs/`. Solución: se agregó un paso de sincronización en `ejecutar_proyecto.py` usando `shutil.copy2()` que copia los tres dashboards de `dashboard/` a `docs/` después de cada regeneración.

6.5 Fase V — Publicación en GitHub Pages

- Se creó el repositorio público `analisis-climatico-fotovoltaico` en la cuenta `fernansaurio` de GitHub.
- Se configuró GitHub Pages para servir desde la carpeta `/docs` de la rama `master`.
- La autenticación SSH falló por incompatibilidad de llaves; se resolvió usando HTTPS con token de acceso personal vía `'gh auth token'`.
- Sitio publicado: <https://fernansaurio.github.io/analisis-climatico-fotovoltaico/>

7. Arquitectura del Software

7.1 Estructura de Archivos

Archivo / Directorio	Descripción
ejecutar_proyecto.py	Punto de entrada principal. Orquesta todo el pipeline.
src/analisis_climatico.py	Carga, curado, estadísticos y dashboard climático (~6,400 líneas).
src/analisis_sma.py	Carga SMA, estadísticos solares y dashboard solar (~2,450 líneas)
src/exportar_fusion.py	Fusión WL+SMA y dashboard de correlación (~1,050 líneas).
src/generar_reporte_pdf.py	Generación de este reporte académico en PDF con fpdf2.
core_math/AjusteCurvas.*	Biblioteca C++: regresión lineal y polinomial.
core_math/MetodosRaices.*	Biblioteca C++: Newton-Raphson, bisección, secante.
core_math/AlgebraLineal.*	Biblioteca C++: Pearson, sistemas lineales, Gauss.
datos_crudos/weatherlink/	Archivos CSV de las estaciones 7GT-EEP y 7GT-UES.
datos_crudos/sma/	Archivos CSV del sistema SMA (inversores y piranómetro).
dashboard/	Dashboards HTML generados (directorio de trabajo local).
docs/	Dashboards HTML para GitHub Pages (sincronizado auto.).
analisis_cache.pkl	Caché MD5 con datos procesados para ejecuciones rápidas.

7.2 Flujo de Ejecución (ejecutar_proyecto.py)

- [1] Verificación de bibliotecas C++ (compilación y prueba de funciones básicas).
- [2] Carga de datos WeatherLink: lectura CSV, interpolación, concatenación de estaciones.
- [3] Carga de datos SMA: lectura CSV, procesamiento de inversores, cálculo de totales.
- [4] Motor estadístico C++: regresión, correlación y parámetros del modelo predictivo.
- [5] Exportación JSON por rangos de fecha para el dashboard de fusión.
- [6] Generación dashboard_fusion.html con datos sincronizados y gráficas Canvas 2D.
- [7] Generación dashboard_solar.html con calendario, gráficas uPlot y estadísticos.
- [8] Generación dashboard_msn_interactivo.html: calendario, rosa de vientos, modelo.
- [9] Sincronización docs/ ← dashboard/ para actualizar GitHub Pages automáticamente.
- [10] Apertura de los tres dashboards en el navegador para verificación.

7.3 Tecnologías Utilizadas

Tecnología	Uso en el proyecto
Python 3.12	Lenguaje principal de análisis y generación de dashboards.
C++ (g++)	Implementación de métodos numéricos de alto rendimiento.
ctypes	Interfaz Python-C++ para funciones de las bibliotecas .so.
pandas	Lectura de CSV y DataFrames (solo E/S, no estadísticos).
numpy	Arrays para indexado y máscaras (no estadísticos).

matplotlib	Figuras estáticas: rosa de vientos, histograma, boxplot, Pearson.
fpdf2	Generación de este reporte en PDF con fuentes Unicode.
uPlot	Gráficas de series temporales interactivas (HTML dashboards).
Chart.js	Gráficas de barras y circulares (HTML dashboards).
GitHub Pages	Publicación web de los dashboards HTML sin servidor.

8. Conclusiones

1. La implementación manual de los métodos numéricos (media, varianza, percentiles, interpolación lineal, correlación de Pearson, regresión lineal y polinomial) sin funciones de alto nivel fue técnicamente viable y permitió comprender en profundidad los algoritmos subyacentes y sus casos límite.
2. El sistema de caché basado en hash MD5 resultó esencial para la eficiencia del pipeline: reduce el tiempo de análisis de 70-90 segundos a menos de 5 segundos en ejecuciones subsecuentes sin cambios en los datos de entrada.
3. La integración de bibliotecas C++ vía ctypes demostró ser una estrategia efectiva para combinar la flexibilidad de Python con el rendimiento de C++ en operaciones numéricas intensivas sobre conjuntos de datos grandes (>130,000 registros).
4. La estación EEP (San Luis Talpa, costera) presenta temperaturas y radiación solar sistemáticamente superiores a la UES (San Salvador, urbana), con alta correlación inter-estacional ($r > 0.90$ en temperatura), validando la coherencia de los datos.
5. El sistema solar fotovoltaico de la EIE muestra alta dependencia de la irradiancia: la correlación entre irradiancia y potencia AC supera $r = 0.85$ en horas de sol, y los meses de estación seca presentan la mayor producción energética.
6. Los dashboards HTML interactivos generados íntegramente desde Python ofrecen una interfaz funcional y visualmente apropiada para la presentación académica de resultados, sin requerir framework web externo.
7. La publicación en GitHub Pages garantiza la reproducibilidad y el acceso público a los resultados sin infraestructura de servidor adicional.

9. Referencias y Recursos

- Davis Instruments. (2024). WeatherLink API Documentation. Davis WeatherLink.
- SMA Solar Technology AG. (2024). Sunny WebBox — Manual de usuario.
- Quarteroni, A., Saleri, F., & Gervasio, P. (2014). Scientific Computing with MATLAB and Octave (4th ed.). Springer.
- Chapra, S. C., & Canale, R. P. (2015). Numerical Methods for Engineers (7th ed.). McGraw-Hill Education.
- FPDF2 Project. (2024). fpdf2 — Free PDF library for Python. <https://py-pdf.github.io/fpdf2/>
- uPlot. (2024). A small, fast chart library for time series. <https://github.com/leeoniya/uPlot>
- Repositorio del proyecto: <https://github.com/fernansaurio/analisis-climatico-fotovoltaico>

- Sitio publicado: <https://fernansaurio.github.io/analisis-climatico-fotovoltaico/>